

Geometric Background

ROBOTICS — Lezione 3

Fundamental Rotation Matrices

Rotation about the z-axis

$$R_{z(\alpha)} = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Matlab translation zRot:

% Returns the rotation about the z axis

```
function [Rz] = zRot(alpha)
    Rz = [cos(alpha), -sin(alpha), 0;
          sin(alpha),  cos(alpha), 0;
          0, 0, 1];
```

end

Rotation about the y-axis

$$R_{y(\beta)} = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix}$$

```
function [Ry] = yRot(beta)
    Ry = [cos(beta), 0, sin(beta);
          0, 1, 0;
          -sin(beta), 0, cos(beta)];
```

end

Rotation about the x-axis

$$R_{x(\gamma)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{pmatrix}$$

```
function [Rx] = xRot(gamma)
    Rx = [1, 0, 0;
          0, cos(gamma), -sin(gamma);
          0, sin(gamma), cos(gamma)];
```

end

Drawing a reference frame

Script scriptDraw draws a frame and a second frame rotated by 45° about z:

```
close all; view(3); % 2D or 3D plots
daspect([1 1 1]);
plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]); hold on % x axis
plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]); % y axis
plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]); % z axis
v1 = zRot(pi/4) * [1;0;0] % x axis rotated 45 deg around z
v2 = zRot(pi/4) * [0;1;0] % y axis rotated 45 deg around z
v3 = zRot(pi/4) * [0;0;1] % z axis rotated 45 deg around z
plot3([0,v1(1)],[0,v1(2)],[0,v1(3)], 'Color',[1,0,0],
'LineStyle','--');
plot3([0,v2(1)],[0,v2(2)],[0,v2(3)], 'Color',[0,1,0],
'LineStyle','--');
plot3([0,v3(1)],[0,v3(2)],[0,v3(3)], 'Color',[0,0,1],
'LineStyle','--');
```

Composition of Rotations

Composing two rotations

Three frames share the same origin: $O - x_0y_0z_0$, $O - x_1y_1z_1$, $O - x_2y_2z_2$; a point P has coordinates p_0, p_1, p_2 in the three frames. Let R_i^j denote the rotation matrix of Frame i with respect to Frame j :

$$p_1 = R_2^1 p_2 \quad p_0 = R_1^0 p_1 \quad p_0 = R_2^0 p_2$$

By substitution and comparison:

$$p_0 = R_1^0 R_2^1 p_2 \Rightarrow R_2^0 = R_1^0 R_2^1$$

The relation is obtained in two steps: first rotate $O - x_0y_0z_0$ to align it with frame 1 (R_1^0); then rotate again to align frame 1 with frame 2 (R_2^1).

Composition of Rotations A rotation can be expressed as a sequence of partial rotations, each one defined with respect to the previous one. The frame from which the rotation occurs is termed the **current frame**. The sequence is obtained by **post-multiplying** the matrices associated with each rotation.

$$R_i^j = (R_j^i)^{-1} = (R_j^i)^T$$

Example in the current frame

We first rotate by φ about the current y -axis, then by θ about the current z -axis:

$$R_2^0 = R_{y(\varphi)} R_{z(\theta)}$$

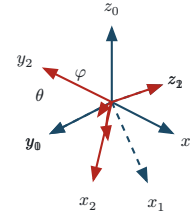


Fig. 1: Rotations about axes in the current frame: φ about the current y -axis, then θ about the current z -axis.

Another example

$$R_4^0 = R_{z(\alpha)} R_{y(\beta)} R_{y(\gamma)} R_{z(\delta)}$$

The order is absolutely important:

$$R_{z(\alpha)} R_{y(\beta)} R_{y(\gamma)} R_{z(\delta)} \neq R_{y(\gamma)} R_{z(\delta)} R_{z(\alpha)} R_{y(\beta)}$$

Rotations about fixed axes

For a generic point P , $p_0 = R_1^0 p_1$. For a generic linear application A defined in frame $O - x_0y_0z_0$:

$$q_0 = A p_0 \quad q_1 = R_0^1 A p_0 = R_0^1 A R_1^0 p_1 = (R_1^0)^{-1} A R_1^0 p_1$$

Example: $R_1^0 = R_{y(\varphi)}$ around the y_0 -axis, and a rotation $A = R_2$ around the z_0 -axis (not z_1 as in the current frame):

$$R_2^0 = R_1^0 R_2 \quad R_2 = (R_1^0)^{-1} R_{z(\theta)} R_1^0$$

$$R_2^0 = R_1^0 (R_1^0)^{-1} R_{z(\theta)} R_1^0 = R_{z(\theta)} R_{y(\varphi)}$$

Fixed axes When the composition happens with **fixed** axes we have to **pre-multiply**, instead of post-multiplying as in the current frame.

$$R_4^0 = R_{z(\delta)} R_{y(\gamma)} R_{y(\beta)} R_{z(\alpha)}$$

Rotation about current axes — example

1. A rotation of θ about the current x -axis. 2. A rotation of φ about the current z -axis.

$$R_2^0 = R_{x(\theta)} R_{z(\varphi)}$$

A point p_2 in frame $O - x_2y_2z_2$ corresponds to $p_0 = R_2^0 p_2$ in the initial frame.

More complex examples

Three rotations: θ about current x , φ about current z , α about fixed z_0 :

$$R_3^0 = R_1^0 R_2^1 R_3 = R_{x(\theta)} R_{z(\varphi)} R_3$$

where R_3 denotes the notation with respect to the initial fixed axes, in the current frame $O - x_2y_2z_2$:

$$R_3 = (R_1^0 R_2^1)^{-1} R_{z(\alpha)} R_1^0 R_2^1 = (R_{x(\theta)} R_{z(\varphi)})^{-1} R_{z(\alpha)} R_{x(\theta)} R_{z(\varphi)}$$

$$\Rightarrow R_3^0 = R_{z(\alpha)} R_{x(\theta)} R_{z(\varphi)}$$

Five rotations: θ about current x , φ about current z , α about fixed z_0 , β about current y , δ about fixed x_0 :

$$R_r^0 = R_{x(\delta)} R_{z(\alpha)} R_{x(\theta)} R_{z(\varphi)} R_{y(\beta)}$$

Check Verify this result yourself with paper and a pencil.

Parametrisation of Rotations

Degrees of freedom

A 3D rotation matrix has 9 elements, but a rotation is completely specified by **three** parameters. In general a rotation is an element of $SO(m)$ completely specified by $\frac{m(m-1)}{2}$ parameters:

- 1 parameter for $SO(2)$; 3 parameters for $SO(3)$.
- A 3D rotation is specified by three rotations about different axes, such that no two adjacent rotations are about the same axis.
- There are $3 \times 2 \times 2 = 12$ parametrisations: 3 choices for the first axis, 2 for the second (rule out the first), 2 for the third (rule out the second).

Euler angles

Euler angles are the angles of three rotations about the current axes. A popular convention is **XYZ rotations about fixed axes**, known in aeronautics as **roll-pitch-yaw**, denoting the change in attitude of an aircraft with respect to three fixed axes. Since the rotation is about fixed axes we have to pre-multiply:

$$R = R_{z(\varphi)} R_{y(\theta)} R_{x(\psi)}$$

ψ = roll angle, θ = pitch angle, φ = yaw angle.

Euler — Matlab

A rotation matrix can be created by specifying three angles; one way is the triple of Euler angles (three rotations about the current axes, ZYZ):

$$R(\varphi, \theta, \psi) = R_{z(\varphi)} R_{y'(\theta)} R_{z''(\psi)}$$

```
function [R] = eulerRot(phi, theta, psi)
    R = zRot(phi) * yRot(theta) * zRot(psi);
end
```

Script `scriptDrawEuler` tests it with $R = \text{eulerRot}(\pi/6, \pi/6, 0)$ and plots the three rotated vectors $R*[1;0;0]$, $R*[0;1;0]$, $R*[0;0;1]$ (same drawing structure as `scriptDraw`).

RPY angles

With the notation $c_\alpha = \cos(\alpha)$, $s_\alpha = \sin(\alpha)$:

$$R = \begin{pmatrix} c_\varphi c_\theta & c_\varphi s_\theta s_\psi - s_\varphi c_\psi & c_\varphi s_\theta c_\psi + s_\varphi s_\psi \\ s_\varphi c_\theta & s_\varphi s_\theta s_\psi + c_\varphi c_\psi & s_\varphi s_\theta c_\psi - c_\varphi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{pmatrix}$$

Given $R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$, the inverse problem has two solutions. For $\theta \in (-\frac{\pi}{2}, \frac{\pi}{2})$:

$$\varphi = \text{atan2}(r_{21}, r_{11}), \quad \psi = \text{atan2}(r_{32}, r_{33}),$$

$$\theta = \text{atan2}(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2})$$

For $\theta \in (\frac{\pi}{2}, 3\frac{\pi}{2})$:

$$\varphi = \text{atan2}(-r_{21}, -r_{11}), \quad \psi = \text{atan2}(-r_{32}, -r_{33}),$$

$$\theta = \text{atan2}(-r_{31}, -\sqrt{r_{32}^2 + r_{33}^2})$$

Exercise Solve the inverse problem and find the two solutions.

RPY — Gimbal lock

An important feature of RPY (and generally of Euler-based representations) is that the three angles act as three independent degrees of freedom. At a pitch angle of $\pm\frac{\pi}{2}$ the roll and yaw rotation axes become aligned, resulting in the loss of a degree of freedom: a rotation around one axis can be replicated by a rotation around the other. Indeed, when $\theta = \frac{\pi}{2}$:

$$R = \begin{pmatrix} c_\varphi c_\theta & c_\varphi s_\theta s_\psi - s_\varphi c_\psi & c_\varphi s_\theta c_\psi + s_\varphi s_\psi \\ s_\varphi c_\theta & s_\varphi s_\theta s_\psi + c_\varphi c_\psi & s_\varphi s_\theta c_\psi - c_\varphi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{pmatrix} = \begin{pmatrix} 0 & s(\varphi + \psi) & c(\varphi + \psi) \\ 0 & c(\varphi + \psi) & -s(\varphi + \psi) \\ -1 & 0 & 0 \end{pmatrix}$$

From this, φ and ψ do not act independently: all rotations achieved by a combination of φ and ψ can be accomplished by a single rotation of angle $(\varphi + \psi)$. The inverse mapping from the rotation matrix to RPY angles is singular at this point: infinitely many combinations of φ and ψ produce the same matrix.

- Profound reason: it is like creating a 2D map from a sphere; when flattening it, the poles become lines, so a single point on the sphere maps to an infinite number of points.
- In the singular configuration small changes in physical orientation produce discontinuous jumps in the calculated RPY angles.

Robotics issue This lack of invertibility is a major problem for physical systems like robots, which require smooth and continuous control.

Axis-angle representation

A redundant parametrisation uses an axis $k = [k_x, k_y, k_z]$ with $|k| = 1$ and an angle θ , four parameters overall. The rotation $R_{k(\theta)}$ is found by transforming coordinates: find a new frame $O - x_1 y_1 z_1$ with z_1 coincident with k , rotating $O - x_0 y_0 z_0$ by α about the z -axis and by β around the y -axis, so $R_1^0 = R_{z(\alpha)} R_{y(\beta)}$. For $q_1 = A p_1$:

$$q_0 = (R_1^0)^{-1} A R_0^1 p_0 = R_{z(\alpha)} R_{y(\beta)} A (R_{z(\alpha)} R_{y(\beta)})^{-1} \Rightarrow R_{k(\theta)} = R_{z(\alpha)} R_{y(\beta)} R_{z(\theta)} R_{y(-\beta)} R_{z(-\alpha)}$$

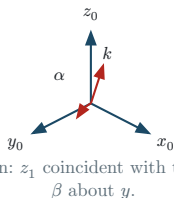


Fig. 2: Axis-angle construction: z_1 coincident with the unit axis k ; α about z_0 and β about y_0 .

Using trigonometry:

$$\sin \alpha = \frac{k_y}{\sqrt{k_x^2 + k_y^2}}, \quad \cos \alpha = \frac{k_x}{\sqrt{k_x^2 + k_y^2}},$$

$$\sin \beta = \frac{k_z}{\sqrt{k_x^2 + k_z^2}}, \quad \cos \beta = \frac{k_x}{\sqrt{k_x^2 + k_z^2}}$$

From Rodrigues' formula:

$$R_{k(\theta)} = \begin{pmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{pmatrix}$$

Four parameters plus the constraint $k_x^2 + k_y^2 + k_z^2 = 1$ give three degrees of freedom. Also $R_{-k}(-\theta) = R_{k(\theta)}$: inverting both axis and angle yields the same rotation.

Unlike RPY, there are no singularities such as Gimbal Lock (it is locally invertible, a local homeomorphism), but the parametrisation is not intuitive, and it is not straightforward to analytically compose or invert subsequent rotations as with matrices.

Quaternions

From $\mathbb{R}$ to $\mathbb{R}^4$

Consider a real number $x \in \mathbb{R}$: $\frac{1}{x} \cdot x = 1 \Rightarrow x^{-1} = \frac{1}{x}$. Consider an element of $\mathbb{R}^2$, $x = \begin{pmatrix} a \\ b \end{pmatrix}$: its inverse exists using complex numbers. Define $z = a + ib$ (i imaginary unit) and the **complex conjugate** $z^* = a - ib$; consider $z_1 = \frac{z^*}{|z|^2}$. Then:

$$z \cdot z_1 = z \frac{z^*}{|z|^2} = 1, \quad |z|^2 = a^2 + b^2, \quad \text{hence } z^{-1} = z_1$$

The same trick cannot be repeated with $\mathbb{R}^3$, but it can with $\mathbb{R}^4$ by using quaternions.

Definitions

Complex number Let 1 and i be the basis elements, with the axiom $i^2 = -1$. A complex number is an element of a vector space over the reals with the form $z = x + iy$, where $x, y \in \mathbb{R}$.

Quaternion Let 1 and i, j, k be the basis elements, with the axioms $i^2 = j^2 = k^2 = ijk = -1$. A quaternion is an element of a vector space over the reals with the form $q = q_0 + q_1 i + q_2 j + q_3 k$. Shorthand distinguishes the **scalar part** q_0 and the **vector part** $q = q_1 i + q_2 j + q_3 k$: $q = q_0 + q$, equivalently $[q_0, q]$.

Background on vector products

Scalar (dot) product Given $u = u_1 i + u_2 j + u_3 k$ and $v = v_1 i + v_2 j + v_3 k$, $u \cdot v = u_1 v_1 + u_2 v_2 + u_3 v_3$. If θ is the angle between the vectors, $u \cdot v = |u| |v| \cos \theta$. It represents the projection of one vector onto the other and is commutative.

Vector (cross) product Given u and v , $s = u \times v$ is a vector orthogonal to both, with $|s| = |u| |v| \sin \theta$. With $s = s_1 i + s_2 j + s_3 k$:

$$s_1 = u_2 v_3 - u_3 v_2, \quad s_2 = u_3 v_1 - u_1 v_3, \quad s_3 = u_1 v_2 - u_2 v_1$$

The vector product is anti-commutative: $u \times v = -v \times u$.

Multiplication of basis elements

Applying $i^2 = j^2 = k^2 = ijk = -1$:

$$ij = k, \quad ji = -k, \quad jk = i, \quad kj = -i, \quad ki = j, \quad ik = -j$$

These rules are the same as those of the vector cross product.

Quaternion product

With $p = p_0 + p$ and $q = q_0 + q$:

$$pq = (p_0 + p)(q_0 + q) = p_0 q_0 + p_0 q + q_0 p + pq$$

Using the multiplication rules, $pq = -p \cdot q + p \times q$, so the full product is:

$$pq = (p_0 q_0 - p \cdot q) + (p_0 q + q_0 p + p \times q)$$

The scalar part is $p_0 q_0 - p \cdot q$ and the vector part is $p_0 q + q_0 p + p \times q$. The product is **associative** but **not commutative**: $pq \neq qp$.

Conjugate, length and inverse

$$q = q_0 + q_1 i + q_2 j + q_3 k \Rightarrow q^* = q_0 - q_1 i - q_2 j - q_3 k$$

$$qq^* = (q_0 + q)(q_0 - q) = q_0^2 - q_0 q + q_0 q - q^2$$

$$= q_0^2 - (-q \cdot q + q \times q) = q_0^2 + |q|^2 = q_0^2 + q_1^2 + q_2^2 + q_3^2$$

Quaternion length The quantity $|q| = \sqrt{qq^*} = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$ is the quaternion length or norm.

Quaternion inverse Every non-zero quaternion has a unique inverse $q^{-1} = \frac{q^*}{|q|^2}$:

$$q \frac{q^*}{|q|^2} = 1$$

For a unit-length quaternion this simplifies to $q^{-1} = q^*$.

Complex numbers and rotations

A complex number in Euler form is $z = \rho(\cos(\theta) + j \sin(\theta))$. Multiplying two:

$$\begin{aligned} z_1 z_2 &= \rho_1(\cos \theta_1 + j \sin \theta_1) \rho_2(\cos \theta_2 + j \sin \theta_2) \\ &= \rho_1 \rho_2 (\cos(\theta_1 + \theta_2) + j \sin(\theta_1 + \theta_2)) \end{aligned}$$

Rotation Multiplying a complex number by a unit-norm (unit-modulus) complex number has the effect of a rotation. Multiplying by its complex conjugate «undoes» the rotation, aligning the result with the real axis.

Unit-length quaternions

Consider a unit-length quaternion $q = [\eta, \varepsilon]$, with scalar part η and vector part $\varepsilon = \begin{pmatrix} \varepsilon_x \\ \varepsilon_y \\ \varepsilon_z \end{pmatrix}$. It can be expressed as:

$$q = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \mathbf{k} \right)$$

where $\mathbf{k}$ is a unit vector:

$$\mathbf{k} = \frac{\varepsilon}{|\varepsilon|}, \quad \theta = 2 \operatorname{atan2}(|\varepsilon|, \eta)$$

Key fact The unit-length quaternion $q = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \mathbf{k} \right)$ represents a rotation of angle θ about the axis defined by the unit vector $\mathbf{k}$.

Quaternions and rotations

Let $q = [\eta, \varepsilon]$ be a unit quaternion and $p = [0, \mathbf{p}]$ a pure quaternion representing a vector $\mathbf{p}$. The rotated vector $\mathbf{p}'$ is computed by conjugation $\mathbf{p}' = q\mathbf{p}q^*$, yielding a pure quaternion $[0, \mathbf{p}']$. After computation:

$$\mathbf{p}' = (\eta^2 - |\varepsilon|^2)\mathbf{p} + 2\eta(\varepsilon \times \mathbf{p}) + 2(\varepsilon \cdot \mathbf{p})\varepsilon$$

Substituting $\eta = \cos(\frac{\theta}{2})$, $\varepsilon = \sin(\frac{\theta}{2})\mathbf{k}$ and $|\varepsilon|^2 = \sin^2(\frac{\theta}{2})$:

$$\begin{aligned} \mathbf{p}' &= \left(\cos^2\left(\frac{\theta}{2}\right) - \sin^2\left(\frac{\theta}{2}\right) \right) \mathbf{p} \\ &+ 2 \cos\left(\frac{\theta}{2}\right) \sin\left(\frac{\theta}{2}\right) (\mathbf{k} \times \mathbf{p}) + 2 \sin^2\left(\frac{\theta}{2}\right) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k} \\ &= \cos(\theta) \mathbf{p} + \sin(\theta) (\mathbf{k} \times \mathbf{p}) + (1 - \cos(\theta)) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k} \end{aligned}$$

Rodrigues' rotation formula The formula

$$\begin{aligned} \mathbf{p}' &= \cos(\theta) \mathbf{p} + \sin(\theta) (\mathbf{k} \times \mathbf{p}) \\ &+ (1 - \cos(\theta)) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k} \end{aligned}$$

describes the rotation of a vector $\mathbf{p}$ by an angle θ about the axis $\mathbf{k}$.

Equivalent rotation matrix

The Rodrigues formula is equivalent to applying a rotation matrix $R_{k(\theta)}$:

$$\mathbf{p}' = R_{k(\theta)} \mathbf{p}$$

$$R_{k(\theta)} = \begin{pmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{pmatrix}$$

with $c_\theta = \cos(\theta)$, $s_\theta = \sin(\theta)$.

Summary

Given $q = [\eta, \varepsilon] = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \mathbf{k} \right)$ associated with a rotation of angle θ about unit vector $\mathbf{k}$:

$$\mathbf{k} = \frac{\varepsilon}{|\varepsilon|}, \quad \theta = 2 \operatorname{atan2}(|\varepsilon|, \eta)$$

The parameters of a unit-length quaternion correspond to three degrees of freedom, expressed as:

- four numbers $\eta, \varepsilon_x, \varepsilon_y, \varepsilon_z$ with the constraint $\eta^2 + \varepsilon_x^2 + \varepsilon_y^2 + \varepsilon_z^2 = 1$;
- four numbers θ, k_x, k_y, k_z with the constraint $k_x^2 + k_y^2 + k_z^2 = 1$.

From rotation matrix to quaternion

Given $R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$:

$$\eta = \frac{1}{2} \sqrt{r_{11} + r_{22} + r_{33} + 1}$$

$$\varepsilon = \frac{1}{2} \begin{pmatrix} \operatorname{sgn}(r_{32} - r_{23}) \sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \operatorname{sgn}(r_{13} - r_{31}) \sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \operatorname{sgn}(r_{21} - r_{12}) \sqrt{r_{33} - r_{11} - r_{22} + 1} \end{pmatrix}$$

$$\operatorname{sgn}(x) = 1 \text{ if } x \geq 0, \quad -1 \text{ if } x < 0$$

Composition via quaternions

Consider two rotations with unit-length quaternions q_0 and q_1 ; the first maps $\mathbf{p}$ to $\mathbf{p}'$, the second maps $\mathbf{p}'$ to $\mathbf{p}''$. With pure quaternions $p = [0, \mathbf{p}]$, $p' = [0, \mathbf{p}']$, $p'' = [0, \mathbf{p}'']$:

$$\mathbf{p}' = q_0 \mathbf{p} q_0^*$$

$$\mathbf{p}'' = q_1 \mathbf{p}' q_1^* = q_1 (q_0 \mathbf{p} q_0^*) q_1^*$$

$$= (q_1 q_0) \mathbf{p} (q_1^* q_0^*) = (q_1 q_0) \mathbf{p} (q_1 q_0)^*$$

The last step uses $(q_1 q_0)^* = q_0^* q_1^*$ (conjugate of a product = product of conjugates in reverse order). The composition of two rotations is thus the product of their quaternions.

Operations and advantages

Operation	Rotation matrices	Unit-quaternions
Inversion	$R^{-1} = R^\top$	$q^{-1} = (\eta, -\varepsilon)$
Composition	$R = R_1 R_2$	$q = q_1 q_2$
Composition formula	—	$q_1 q_2 = (\eta_1 \eta_2 - \varepsilon_1 \cdot \varepsilon_2, \eta_1 \varepsilon_2 + \eta_2 \varepsilon_1 + \varepsilon_1 \times \varepsilon_2)$

- **Inversion:** the inverse of a unit-length quaternion is its conjugate (negate the vector part), much faster than transposing and inverting a matrix.
- **Composition:** two rotations combine by a simple quaternion product, more efficient and numerically stable than matrix multiplication.

In summary

- **Compact representation:** four numbers instead of the nine of a rotation matrix.
- **Absence of Gimbal Lock:** continuous, non-singular representation, unlike Euler angles; significant in computer graphics and robotics.
- **Efficient composition:** simple quaternion multiplication, more efficient than matrix multiplication.
- **Direct interpolation:** smooth constant-velocity interpolation between rotations (spherical linear interpolation, *slerp*), crucial for animations.
- **Non-uniqueness:** a single rotation can be represented by two quaternions, q and $-q$; a minor redundancy that does not demean the practical benefits.

Homogeneous Transformations

Position of a frame in 3D

A rigid body's pose is defined by the position and orientation of a frame attached to it. So far only rotation was considered, ignoring translation (frames with coincident origins); we now return to the general case with both rotation and translation. A point p_1 in frame $\{1\}$ is expressed in frame $\{0\}$ as:

$$p_0 = o_0^1 + R_1^0 p_1$$

Direct and inverted transformation

The inverse is found by isolating p_1 :

$$\begin{aligned} R_1^0 p_1 = p_0 - o_0^1 &\Rightarrow p_1 = (R_1^0)^{-1} (p_0 - o_0^1) = R_1^{0\top} (p_0 - o_0^1) \\ &= R_0^1 (p_0 - o_0^1) = R_0^1 p_0 - R_0^1 o_0^1 \end{aligned}$$

This representation is cumbersome: the inverse involves both a matrix transpose and a matrix-vector product.

Homogeneous coordinates (1)

A compact way to represent rotation and translation: create the homogeneous representation $\tilde{p}$ of p by appending a coordinate of value 1, $\tilde{p} = \begin{pmatrix} p \\ 1 \end{pmatrix}$, and construct a 4×4 homogeneous transformation matrix:

$$A_0^1 = \begin{pmatrix} R_1^0 & o_0^1 \\ 0^\top & 1 \end{pmatrix}$$

The transformation of a point from frame $\{1\}$ to frame $\{0\}$ is then a single matrix-vector multiplication: $\tilde{p}_0 = A_0^1 \tilde{p}_1$.

Homogeneous coordinates (2)

The coordinate transformation is a simple matrix multiplication $\tilde{p}_0 = A_0^1 \tilde{p}_1$; the inverse is found by inverting the matrix:

$$\tilde{p}_1 = A_1^0 \tilde{p}_0 = (A_0^1)^{-1} \tilde{p}_0$$

$$A_1^0 = (A_0^1)^{-1} = \begin{pmatrix} R_1^{0\top} & -R_1^{0\top} o_0^1 \\ 0^\top & 1 \end{pmatrix}$$

Homogeneous coordinates (3)

- Unlike rotation matrices, homogeneous transformations are not orthogonal: for a general homogeneous matrix A , $A^{-1} \neq A^\top$.
- The formalism generalizes the properties of the rotation group to include translations.
- A sequence of transformations in the current axes is a simple matrix product: $\tilde{p}_0 = A_0^1 A_1^2 \dots A_{n-1}^n \tilde{p}_n$.
- The set of all homogeneous transformations forms a group called the **special Euclidean group** $SE(3)$, the semi-direct product $SE(3) = \mathbb{R}^3 \rtimes SO(3)$.

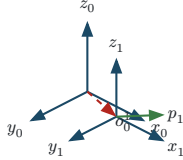


Fig. 3: Position of a frame in 3D: $p_0 = o_0^1 + R_1^0 p_1$.

Matlab Examples for Homogeneous Transformations

homogeneousTrans

An homogeneous transformation matrix $A = \begin{pmatrix} R(\varphi, \theta, \psi) & O_1^0 \\ O & 1 \end{pmatrix}$:

```
% homogeneous transformation
function [T] = homogeneousTrans(phi, theta, psi, Ov)
    R = eulerRot(phi, theta, psi);
    T = [R, Ov;
        zeros(1,3), 1];
end
```

Test scripts

scriptDrawHom uses the function and plots the translated/rotated frame:

```
T = homogeneousTrans(0, 0, pi/4, [1;1;1]);
O1 = T * [0;0;0;1];
v1 = T * [1;0;0;1]; v2 = T * [0;1;0;1]; v3 = T * [0;0;1;1];
scriptDrawHom1 uses built-in Matlab functions instead:
T = makehgtform('translate',[1,1,1], 'zrotate', pi/4);
```

Drawing reference frames

scriptTestTriad uses a community file triad to draw axis frames:

```
axs = axes; view(3); daspect([1 1 1]);
h = triad('Parent', axs); % frame coincident with the current axes
h1 = triad('Parent', h, 'Matrix', ...
    makehgtform('translate',[1,1,1], 'xrotate', pi/4), ...
    'linewidth', 3, 'linestyle', '--');
```

Matlab transformation objects

Matlab provides an object to create a transformation, link it to the current axis, and define child objects transformed accordingly.

```
scriptTransformTest:
axs = axes('XLim', [-1.5 1.5], 'YLim', [-1.5 1.5], 'ZLim', [-1.5 1.5]);
view(3); grid on;
t = hgtransform('Parent', axs); % empty transformation
[x,y,z] = cylinder([1,1], 100);
h = surface(x,y,z, 'FaceColor', 'red');
set(h, 'Parent', t); % link cylinder to t
pause
Rz = makehgtform('xrotate', 2*pi/3); % rotation matrix
set(t, 'Matrix', Rz); % apply to t and children
drawnow; % update children objects
scriptTransformTriads draws two frames and animates the second by rotating about z:
t = hgtransform('Parent', axs); h = triad('Parent', t);
h1 = triad('Parent', h, 'Matrix', ...
    makehgtform('translate',[1,1,1], 'xrotate', pi/4));
for angle = 0:0.5:360
    Rz = makehgtform('zrotate', deg2rad(angle));
    set(t, 'Matrix', Rz); drawnow;
end
```

Some questions for you

- Rotation matrix about the x -axis by γ : $R_{x(\gamma)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{pmatrix}$
- Difference current vs fixed axes: **current axes** apply each successive rotation relative to the moving frame (post-multiplication); **fixed axes** apply rotations relative to the original fixed frame (pre-multiplication).
- Matlab rotation about the y -axis: $yRot(beta)$ with $R_y = [\cos(beta), 0, \sin(beta); 0, 1, 0; -\sin(beta), 0, \cos(beta)]$.
- A vector rotated first by $R_{y(\varphi)}$ then by $R_{z(\theta)}$: $R = R_{y(\varphi)} R_{z(\theta)}$.

- Order matters because rotation matrices do not generally commute: $R_a R_b \neq R_b R_a$; changing the order changes the final orientation.
- For fixed axes we pre-multiply because each new rotation is applied relative to the original fixed frame.
- A 3D rotation has **3** independent parameters: a 3×3 matrix (9 values) is reduced by orthogonality and unit determinant to 3 degrees of freedom.
- Euler angles are a sequence of 3 rotations about axes; in aeronautics: roll (ψ), pitch (θ), yaw (φ) with fixed axes, $R = R_{z(\varphi)} R_{y(\theta)} R_{x(\psi)}$.
- Gimbal lock in RPY: at pitch $\theta = \pm \frac{\pi}{2}$ the roll and yaw axes align, losing one degree of freedom; different angle sets map to the same rotation.
- Axis-angle form: a unit vector $\mathbf{k} = (k_x, k_y, k_z)$ and an angle θ , giving $R_{\mathbf{k}(\theta)}$.
- Rodrigues' rotation formula: $\mathbf{p}' = \cos(\theta)\mathbf{p} + \sin(\theta)(\mathbf{k} \times \mathbf{p}) + (1 - \cos(\theta))(\mathbf{k} \cdot \mathbf{p})\mathbf{k}$.
- Axis-angle avoids gimbal lock because it locally represents rotations without singularities: no two axes can align to cause a degree-of-freedom loss.
- Quaternion: $q = q_0 + q_1 i + q_2 j + q_3 k = [q_0, \mathbf{q}]$; q_0 scalar part, $\mathbf{q} = (q_1, q_2, q_3)$ vector part.
- Conjugate $q^* = q_0 - q_1 i - q_2 j - q_3 k$; inverse $q^{-1} = \frac{q^*}{|q|^2}$ (for unit quaternions $q^{-1} = q^*$).
- If q_1, q_2 represent rotations, their composition is $q = q_1 q_2$; applied to $\mathbf{p}$: $\mathbf{p}' = q_1(q_2 p q_2^*) q_1^* = (q_1 q_2) p (q_1 q_2)^*$.
- Two advantages over Euler angles in robotics: avoid gimbal lock (non-singular representation); efficient composition and interpolation (slerp).
- Homogeneous coordinates represent rotation and translation in a single 4×4 matrix.
- General homogeneous transformation matrix: $A = \begin{pmatrix} R & o \\ 0^\top & 1 \end{pmatrix}$ with R rotation matrix and o translation vector.
- Inverse of a homogeneous transformation: $A^{-1} = \begin{pmatrix} R^\top & -R^\top o \\ 0^\top & 1 \end{pmatrix}$.
- Homogeneous transformations form the Special Euclidean group $SE(3) = \mathbb{R}^3 \rtimes SO(3)$.